# Machine Learning for Physicists

Tutorials

Wifi password: Mpl4guestS!

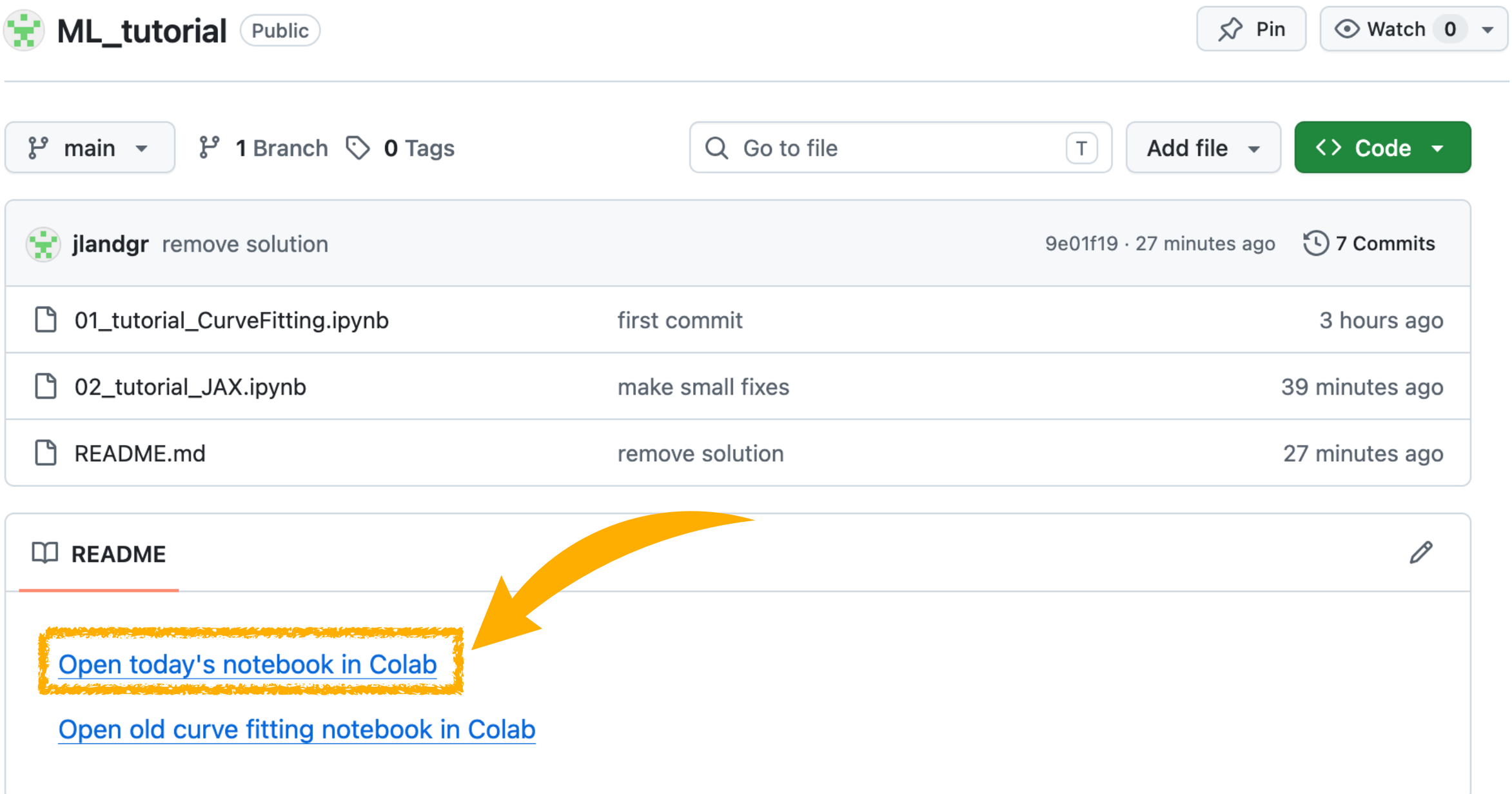
Jonas Landgraf
Max Planck Institute for the Science of Light

(Image generated by a net with 20 hidden layers)

# Practice session: Using JAX for optimization

Machine Learning for Physicists

see: https://github.com/jlandgr/ML_tutorial




Alternative: Download 02_tutorial_JAX.ipynb and run it lokally on your laptop

# **Reminder:** How to use Jax for simple optimization

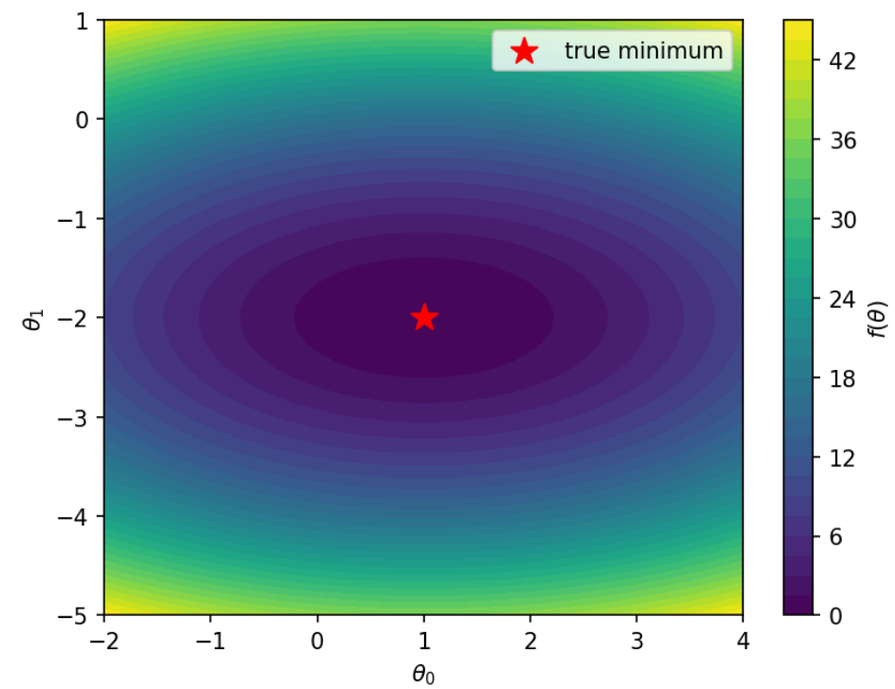
Task: Find the minimum of a function f($\theta$)

# **Reminder:** How to use Jax for simple optimization

Task: Find the minimum of a function f($\theta$)

```
def f(theta):
    return (theta[0] - 1.0)**2 + 4.0 * (theta[1] + 2.0)**2

theta_star = jnp.array([1.0, -2.0])
```

✓ 0.2s



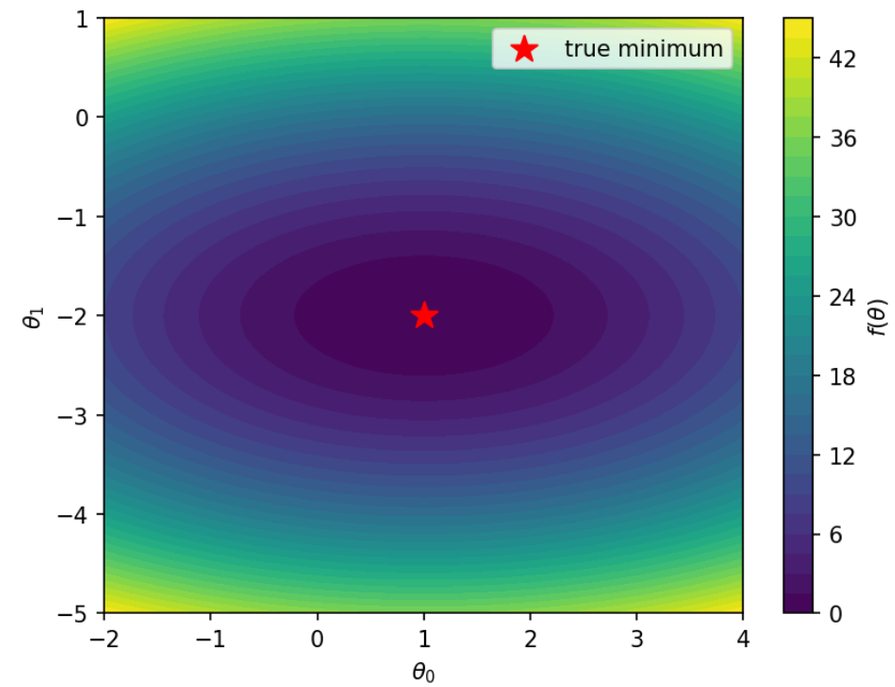
# **Reminder:** How to use Jax for simple optimization

Task: Find the minimum of a function f($\theta$)

```
def f(theta):
    return (theta[0] - 1.0)**2 + 4.0 * (theta[1] + 2.0)**2

theta_star = jnp.array([1.0, -2.0])
```

✓ 0.2s

```
# the function grad_f will calculate the gradient of f via automatic differentiation
grad_f = jax.grad(f)

# for comparison, we have implemented the analytical expression for the gradient
def grad_f_analytic(theta):
    return jnp.array([2.0 * (theta[0] - 1.0), 8.0 * (theta[1] + 2.0)])
```



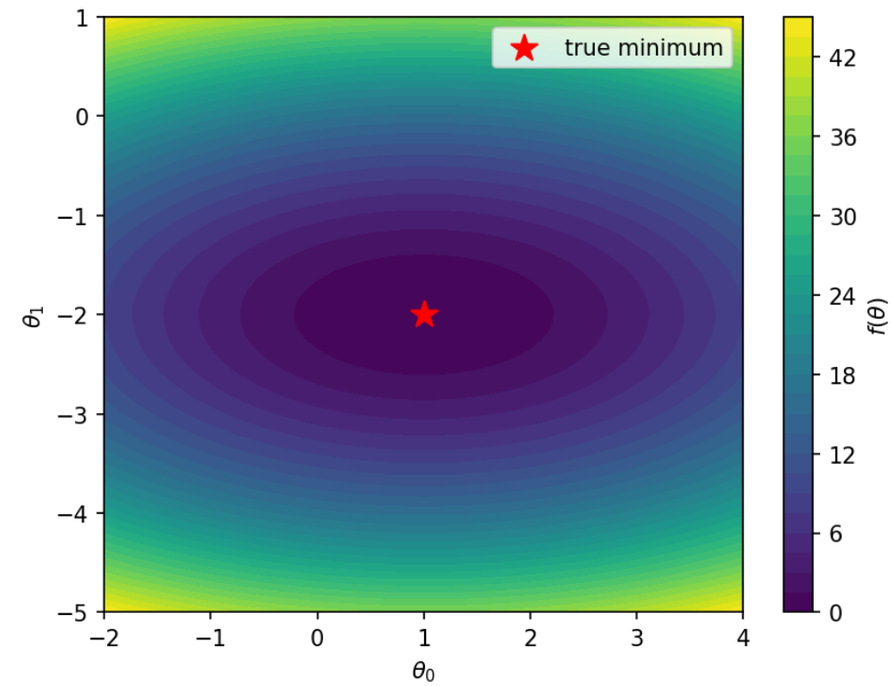
# **Reminder:** How to use Jax for simple optimization

Task: Find the minimum of a function f($\theta$)

```
def f(theta):
    return (theta[0] - 1.0)**2 + 4.0 * (theta[1] + 2.0)**2

theta_star = jnp.array([1.0, -2.0])
```

✓ 0.2s

```
# the function grad_f will calculate the gradient of f via automatic differentiation
grad_f = jax.grad(f)

# for comparison, we have implemented the analytical expression for the gradient
def grad_f_analytic(theta):
    return jnp.array([2.0 * (theta[0] - 1.0), 8.0 * (theta[1] + 2.0)])
```

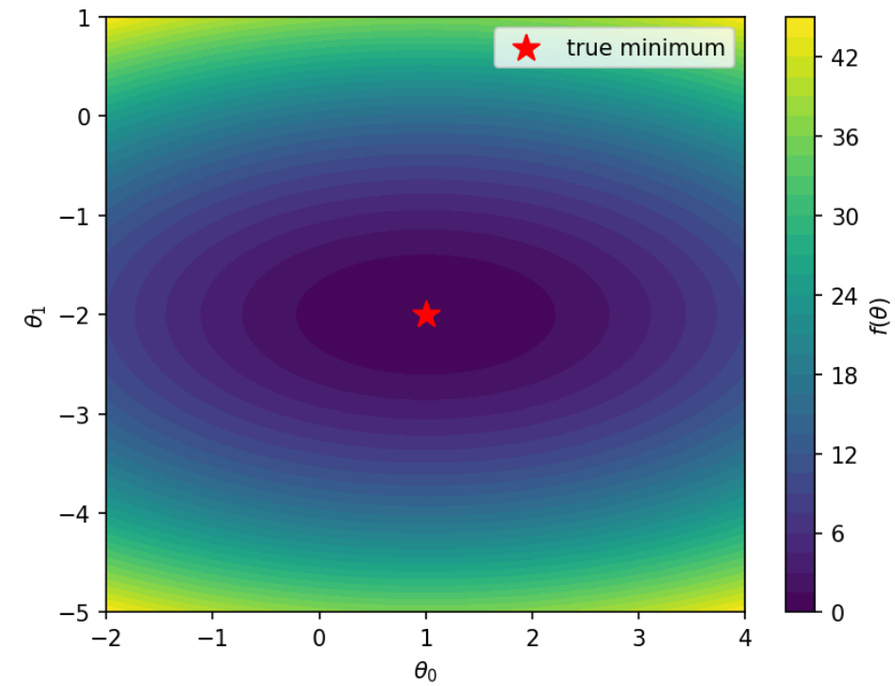


# **Reminder:** How to use Jax for simple optimization

Task: Find the minimum of a function f($\theta$)

```python
def f(theta):
    return (theta[0] - 1.0)**2 + 4.0 * (theta[1] + 2.0)**2

theta_star = jnp.array([1.0, -2.0])
```

✓ 0.2s




```python
# the function grad_f will calculate the gradient of f via automatic differentiation
grad_f = jax.grad(f)

# for comparison, we have implemented the analytical expression for the gradient
def grad_f_analytic(theta):
    return jnp.array([2.0 * (theta[0] - 1.0), 8.0 * (theta[1] + 2.0)])
```

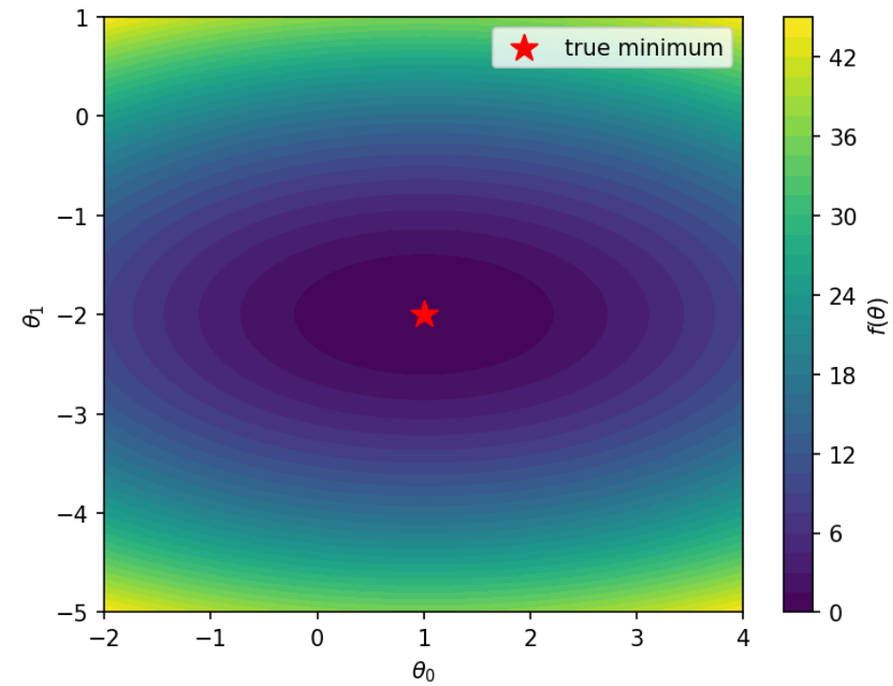
```python
# Select an optimizer (try both!):
optimizer = optax.sgd(learning_rate=0.1)
# optimizer = optax.adam(learning_rate=0.1)

theta = jnp.array([-1.0, 0.5])     # arbitrary starting point
opt_state = optimizer.init(theta)
nsteps = 50

trajectory = [theta]
loss = [f(theta)]

for n in range(nsteps):
    grads = grad_f(theta)
    updates, opt_state = optimizer.update(grads, opt_state)
    theta = optax.apply_updates(theta, updates)
    trajectory.append(theta)
    loss.append(f(theta))

trajectory = jnp.array(trajectory)
loss = jnp.array(loss)
```

# **Reminder:** How to use Jax for simple optimization

Task: Find the minimum of a function f($\theta$)

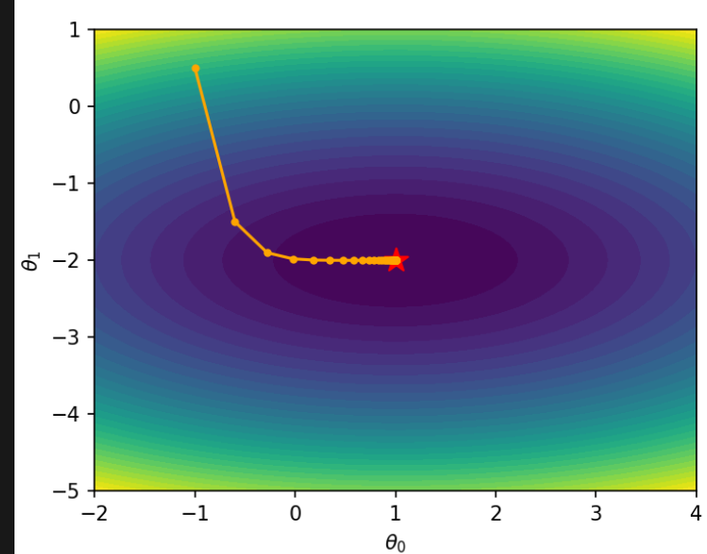
```python
def f(theta):
    return (theta[0] - 1.0)**2 + 4.0 * (theta[1] + 2.0)**2

theta_star = jnp.array([1.0, -2.0])
```

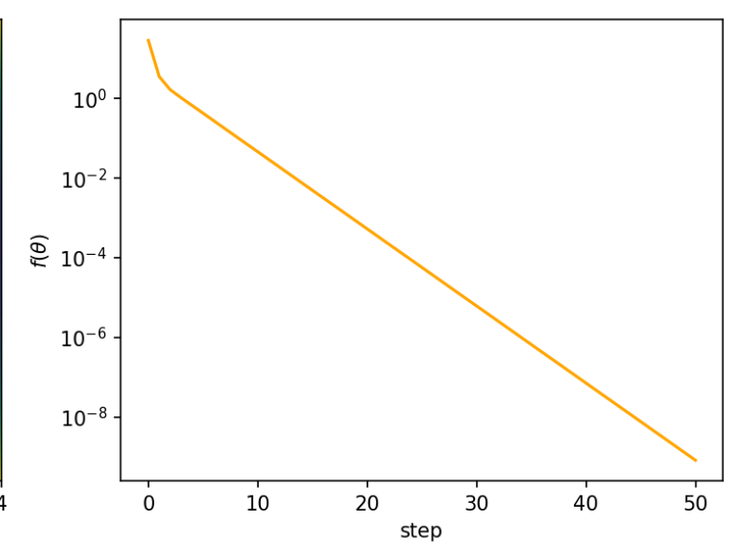
✓ 0.2s




```python
# the function grad_f will calculate the gradient of f via automatic differentiation
grad_f = jax.grad(f)

# for comparison, we have implemented the analytical expression for the gradient
def grad_f_analytic(theta):
    return jnp.array([2.0 * (theta[0] - 1.0), 8.0 * (theta[1] + 2.0)])
```

```python
# Select an optimizer (try both!):
optimizer = optax.sgd(learning_rate=0.1)
# optimizer = optax.adam(learning_rate=0.1)

theta = jnp.array([-1.0, 0.5])     # arbitrary starting point
opt_state = optimizer.init(theta)
nsteps = 50

trajectory = [theta]
loss = [f(theta)]

for n in range(nsteps):
    grads = grad_f(theta)
    updates, opt_state = optimizer.update(grads, opt_state)
    theta = optax.apply_updates(theta, updates)
    trajectory.append(theta)
    loss.append(f(theta))

trajectory = jnp.array(trajectory)
loss = jnp.array(loss)
```



# Tasks:

1. Go through part I of the notebook and remind yourself about the basic Jax functionalities

2. Use Jax to fit the Lorentzian function!
   (Feel free to have a look at part I and the previous notebook 01_tutorial_CurveFitting.ipynb)

3. Apply your code to fit the sinussoidal and three-parameter Lorentzian curves!

Bonus: Use jax.jit to compile functions, and jax.vmap to vectorize the function evaluation!
(Feel free to ask if you need a reminder how the work!)